

개인화 동적 기술트리 기반 커리어 가이드 서비스 「자람이(Zarami)」 통합 제품 상세 기획서 (PRD)

Next.js 15 & Supabase 기반 MVP 완결판 마스터 버전

1. 프로젝트 정의 및 핵심 가치

본 프로젝트 '자람이(Zarami)'는 주니어 프론트엔드 개발자가 미들/시니어 레벨로 성장하는 과정에서 겪는 '학습 방향성 상실' 문제를 해결하기 위한 개인화 동적 로드맵 플랫폼이다. 기존의 정적인 로드맵 서비스(roadmap.sh 등)가 모든 유저에게 일방적이고 동일한 정보를 나열하는 한계를 극복하고, 유저가 보유한 기존 스택을 파악하여 '아는 지식은 영리하게 건너뛰고 (Skip)', 실무 채용 시장 데이터를 반영한 '가장 트렌디한 다음 행동(Next Action)'을 게임화된 UX(식물 키우기 서사)로 제공한다.

💡 이직 포트폴리오를 위한 차별화 전략

단순 기술 나열용 토이 프로젝트와의 차별화를 위해 다음 4대 요소를 기술적으로 증명한다.

- **비용 효율성:** LLM API의 무분별한 호출을 제한하는 데이터 캐싱(Caching) 및 배치 아키텍처.
- **UX 완성도:** React Flow 인터랙티브 캔버스를 활용한 대용량 노드 렌더링 최적화 및 Viewport Lock.
- **인터랙션 정합성:** 낙관적 업데이트(Optimistic Update)와 비회원 로컬 데이터 마이그레이션 파이프라인 구축.
- **제품 중심 엔지니어링:** 실무 채용 공고(Wanted/Jumpit) 텍스트 마이닝 가중치 점수 및 게임화(Gamification) 요소 적용.

2. 시스템 아키텍처 및 데이터 파이프라인

인프라 관리 공수를 제로화하고 프론트엔드 역량을 극대화하기 위해 Next.js 15 App Router와 Supabase Serverless 인프라를 채택한다. 백엔드 API 레이어는 Next.js의 Route Handlers를 활용해 BFF(Backend For Frontend) 패턴으로 구현한다.

레이어	적용 기술 스택	역할 및 특징
Frontend (Core)	Next.js 15 (App Router), TypeScript, Tailwind CSS	Server Component를 활용한 초기 로드맵 데이터 고속 렌더링 및 SEO 대응.
State & View	Zustand, React Flow v11	무한 캔버스 내 노드/엣지 상태 관리 및 유저 컴플리션 상태 실시간 동기화.
Backend & DB	Supabase (PostgreSQL, GoTrue Auth)	별도 백엔드 서버 없이 가입/인증, RDB 스킴 테이블 간의 재귀적 관계 연산 처리.
Data Integration	Node-fetch, Gemini / Claude API	주기적 채용 데이터 스크래핑 및 LLM 기반 키워드 매핑/퀘스트 자동 생성 템플릿 처리.

3. 정밀 데이터베이스 스키마 설계 (PostgreSQL)

기술들 간의 인과 및 계층 구조(부모-자식)를 표현하기 위한 Self-referencing 테이블과 유저 다대다(M:N) 매핑 테이블 구조이다.

3.1. `skills` (마스터 기술 노드 테이블)

```
CREATE TABLE skills (  
  id VARCHAR(50) PRIMARY KEY,          -- 예: 'nextjs', 'zustand'  
  title VARCHAR(100) NOT NULL,          -- 예: 'Next.js', 'Zustand'  
  parent_id VARCHAR(50) REFERENCES skills(id) ON DELETE SET NULL, -- 부모 노드 (Self-reference)  
  category VARCHAR(50) NOT NULL,        -- 예: 'Framework', 'State Management', 'Language'  
  trend_score VARCHAR(10) DEFAULT 'Low', -- High, Medium, Low (채용 데이터 집계 가중치)  
  description TEXT,                      -- 노드 한 줄 요약 정보  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_skills_parent_id ON skills(parent_id);
```

3.2. `user\_skills` (유저별 학습 및 보유 상태 테이블)

```
CREATE TABLE user_skills (  
  id BIGSERIAL PRIMARY KEY,  
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,  
  skill_id VARCHAR(50) REFERENCES skills(id) ON DELETE CASCADE,  
  is_completed BOOLEAN DEFAULT FALSE, -- 유저의 마스터 여부  
  updated_at TIMESTAMP DEFAULT NOW(),  
  UNIQUE(user_id, skill_id)  
);  
  
CREATE INDEX idx_user_skills_composite ON user_skills(user_id, is_completed);
```

3.3. `quests` (기술 노드별 실무 미션 캐싱 테이블)

```
CREATE TABLE quests (  
  id BIGSERIAL PRIMARY KEY,  
  skill_id VARCHAR(50) REFERENCES skills(id) ON DELETE CASCADE UNIQUE,  
  mission_title VARCHAR(255) NOT NULL, -- LLM이 빌드한 핵심 미션 제목  
  checklists JSONB NOT NULL,           -- 검증용 항목 배열 (e.g. ["Zustand 스토어 분리", "구독 최적화"])  
  reference_urls JSONB,                 -- 커스텀 검색 및 큐레이션된 블로그/기술블로그 링크 배열
```

```
created_at TIMESTAMP DEFAULT NOW()  
);
```

4. 핵심 비즈니스 로직 및 알고리즘 구현 가이드

4.1. 기술 스킵(Skip) 및 하위 트리 연쇄 완료 알고리즘

온보딩 시 사용자가 "나 React 마스터야"라고 체크하면, React를 배우기 위해 선행되어야 하는 하위 기초 스킬들(HTML, CSS, JavaScript 기초 등)을 트리 역방향으로 탐색하여 자동으로 `is_completed = true` 처리해 주는 PostgreSQL 재귀 쿼리 (CTE) 로직이다.

```
WITH RECURSIVE skill_ancestors AS (  
  SELECT id, parent_id FROM skills WHERE id = 'react' -- 사용자가 선택한 스택  
  UNION ALL  
  SELECT s.id, s.parent_id FROM skills s  
  INNER JOIN skill_ancestors sa ON s.id = sa.parent_id  
)  
SELECT id FROM skill_ancestors;
```

4.2. 채용 데이터 가중치 기반 트렌드 갱신 파이프라인

유저 요청 시 발생하는 실시간 비용을 차단하기 위해 '주간 배치 파이프라인'을 가동한다. GitHub Actions 크론탭을 활용해 매주 월요일 새벽 채용 요건 텍스트를 크롤링한 뒤, LLM API로 키워드를 스코어링하여 빈도별로 High(🔥 Hot 배지), Medium, Low 가중치를 `skills` 테이블에 캐싱한다.

5. 고도화 세부 명세 (4대 디테일 반영)

5.1. 전역 상태 및 낙관적 업데이트 (Optimistic Update)

- **낙관적 업데이트 흐름:** 유저가 드로워 내 [물주기]를 클릭하면, Zustand Store 상에서 해당 skillId의 is_completed를 즉시 true로 변경하여 화면 노드를 푸른 잎사귀 형태로 UI Re-render 처리한다. 이후 백엔드 API 호출 결과 실패 처리가 감지되면 즉시 이전 상태(false)로 롤백하고 네트워크 에러 토스트를 연동한다.
- **캔버스 뷰포트 고정 (Viewport Lock):** 드로워가 활성화된 레이아웃 상태일 때, 배경 캔버스의 오작동을 막기 위해 React Flow 컴포넌트의 panOnDrag 및 zoomOnScroll 프로퍼티를 즉시 false로 강제 락다운한다.

5.2. 게스트 로컬 데이터 보존 및 회원 마이그레이션 (Migration)

- **비회원 로컬 영속화:** 비로그인 상태의 온보딩/학습 완료 내역은 Zustand persist 미들웨어를 통해 브라우저 LocalStorage 내 zarami-guest-store에 격리 보관한다.
- **로그인 시 병합(Merge) 연산:** 소셜 로그인이 성공(`SIGNED\_IN`)하는 즉시, 로컬 스토리지의 guest_completed_skill_ids: string[]를 추출하여 Supabase DB에 upsert 처리를 수행한다. 이때 ON CONFLICT (user_id, skill_id) DO NOTHING 옵션을 지정해 중복 제약조건 충돌을 완전 방어한다.

5.3. 오프라인 네트워크 대책 및 큐잉 (Queueing)

- **오프라인 UX 가이드:** navigator.onLine 상태 변화를 트래킹하여 차단 상태 감지 시 최상단에 동동 알림 배너를 바인딩한다.
- **큐잉 구조:** TanStack Query의 onlineManager 및 mutationCache 혹은 연동하여 오프라인 상황에서 트리거된 '물주기' 트랜잭션을 메모리 큐에 대기(Queueing) 시킨다. 네트워크 재연결 시 자동으로 서버에 Flush 처리하여 정합성을 일치시킨다.

5.4. '자람이' 반려식물 캐릭터 진화 레벨링 명세 (Gamification)

전체 기술 노드 분모 대비 유저가 마스터한 완료 노드 개수를 기반으로 게이밍 요소의 진행률(%)을 도출한다.

성장 레벨	트리 완주율 (\$Progress\$)	캐릭터 에셋 파일	대시보드 슬로건
Lv.1 씨앗	$0\% \leq \text{Progress} \leq 20\%$	zarami_seed.svg	아직 씨앗 상태예요. 물을 주어 싹을 틔워보세요!
Lv.2 새싹	$21\% \leq \text{Progress} \leq 50\%$	zarami_sprout.svg	파릇파릇 싹이 틔었어요! 핵심 스택이 성장합니다.
Lv.3 줄기	$51\% \leq \text{Progress} \leq 80\%$	zarami_stem.svg	줄기가 제법 단단해졌네요. 고급 아키텍처를 향해!
Lv.4 개화	$81\% \leq \text{Progress} \leq 100\%$	zarami_bloom.svg	정원에 멋진 꽃이 피었습니다! 시니어 도약 완료.

6. 최종 화면별 UX 상세 시나리오

6.1. 온보딩 화면 (`/onboarding`)

카드 형태의 멀티 스텝 가이드 레이아웃으로, 연차 선택 결과와 보유한 기존 핵심 기술 스택 다중 체크에 따라 우측 화분에 흙이 채워지는 CSS 렌더링 전환 효과를 바인딩하여 진입 만족도를 제고한다.

6.2. 대시보드 및 로드맵 메인 (`/dashboard`)

React Flow의 Infinite Canvas 레이아웃을 확장 적용한다. 선행 부모 노드가 완료된 즉시 다음 행동 스택 노드에 animate-pulse 보더 효과를 적용하여 가시성을 확보하고, 완료된 노드는 불투명도 40%와 취소선 처리를 맥인다.

6.3. 상세 서랍 컴포넌트 (`/dashboard?node=id`)

우측 35% 그리드를 차지하는 오버레이 슬라이드 드로워를 설계하고 내부에 캐싱된 AI 실무 퀘스트와 체크리스트, 참고 기술 블로그 링크 컬렉션을 마크다운 뷰어로 파싱한다. 물주기 클릭 시 물뿌리개 이펙트를 트리거한다.